20

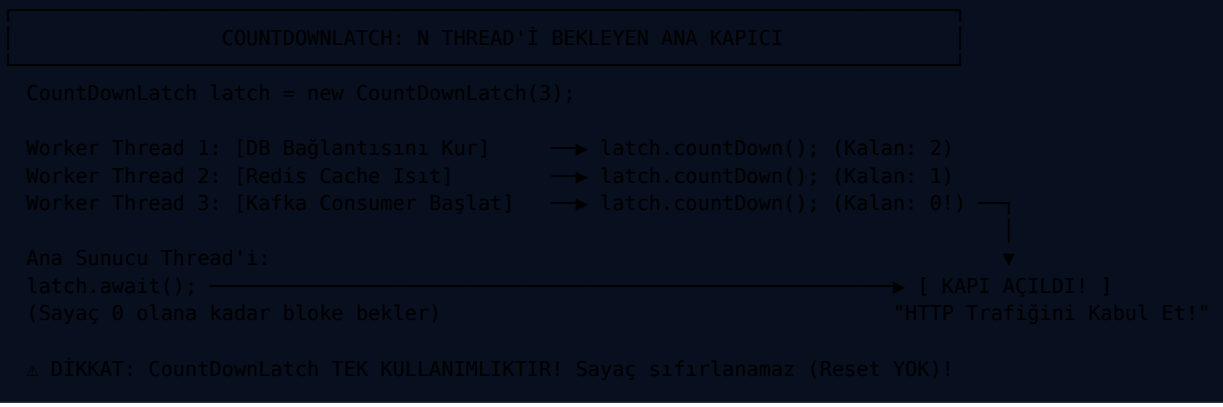
BÖLÜM: İLERI DÜZEY SENKRONIZASYON & KILIT-SIZ ARAÇLAR | MODULE: ADVANCED SYNCHRONIZATION & LOCK-FREE PRIMITIVES

CountDownLatch: Tek Seferlik Geri Sayım Kapıları & Dağıtık Fan-Out

CountDownLatch: One-Shot Countdown Gates for Distributed Fan-Out

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir roket fırlatması düşünün. Ana kumandanın (Main Thread) 'ATEŞLE' düğmesine basması için 3 uzmanın yeşil ışık yakması gerekir: 1. Yakıt uzmanı onaylar (countDown), 2. Motor uzmanı onaylar (countDown), 3. Rota uzmanı onaylar (countDown). Sayaç 3'ten geriye sayar. Sayaç sıfıra ulaştığı anda fırlatma kapısı açılır (`await()` biter) ve roket uzaya fırlar! Bu kilit tek kullanımlıktır; bir kere sıfıra indi mi bir daha kurulamaz!

🇬🇧 ENGLISH (ELI5)

Think of a rocket launch protocol. The mission commander (`main` thread) holds their finger over the ignition switch (`latch.await()`). Three independent engineers must report ready: Telemetry (`countDown()`), Propulsion (`countDown()`), and Life Support (`countDown()`). As the latch decrements from 3 to 0, the gate springs open, releasing the commander to initiate liftoff. CountDownLatch is strictly single-use; once the count hits zero, it cannot be reset.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

CountDownLatch ve AQS Paylaşımlı Durum Mimarisi: 1) AQS Shared Mode: Latch, AQS'in `acquireSharedInterruptibly` ve `releaseShared` metotlarını kullanır. İlk sayaç AQS'in `state` değişkenine yazılır. 2) countDown() Atomikliği: Her `countDown()` çağrısı `state` değişkenini CAS ile 1 azaltır. Sayaç tam 0 olduğu anda, AQS bekleme kuyruğundaki tüm thread'leri `LockSupport.unpark()` ile aynı anda uyandırır. 3) Fan-Out / Fan-In Deseni: MAANG mikromimarisinde bir ana API isteği geldiğinde 5 farklı alt servise (Fiyat, Stok, Kullanıcı, Öneri, Kargo) paralel istek atılır. Ana thread `latch.await(500, TimeUnit.MILLISECONDS)` ile bekler; tüm servisler yanıt verince sonuçlar birleştirilir (Fan-In).

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

CountDownLatch Mechanics in AQS: 1) Shared Synchronizer: Unlike ReentrantLock which acquires exclusive state, CountDownLatch employs AQS Shared Mode. The initial constructor parameter sets the internal AQS `state`. 2) Atomic Decrement: `countDown()` executes CAS loops decrementing `state`. When `state` transitions to 0, it calls `doReleaseShared()`, cascading down the CLH queue unparking every thread blocked on `await()`. 3) One-Shot Invariant: Once `state == 0`, subsequent calls to `countDown()` are inert NOOPs, and calls to `await()` return immediately. It cannot be rewound; reusable synchronization requires `CyclicBarrier`.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: CountDownLatch ile Paralel Mikroservis Fan-Out / Fan-In
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.TimeUnit;

public class MicroserviceAggregator {
    void main() throws InterruptedException {
        int serviceCount = 3;
        var latch = new CountDownLatch(serviceCount);

        IO.println("Ana Servis: Alt servis çağrıları başlatılıyor...");

        // 3 paralel servis çağrısı başlat
        Thread.ofPlatform().name("PriceService").start(() -> {
            try { Thread.sleep(200); IO.println("✔ Fiyat Servisi Tamamlandı"); }
            catch (Exception ignored) {}
            finally { latch.countDown(); }
        });

        Thread.ofPlatform().name("InventoryService").start(() -> {
            try { Thread.sleep(150); IO.println("✔ Stok Servisi Tamamlandı"); }
            catch (Exception ignored) {}
            finally { latch.countDown(); }
        });

        Thread.ofPlatform().name("RecommendationService").start(() -> {
            try { Thread.sleep(300); IO.println("✔ Öneri Servisi Tamamlandı"); }
            catch (Exception ignored) {}
            finally { latch.countDown(); }
        });

        // Maksimum 500ms bekle (SLA Koruması)
        boolean allDone = latch.await(500, TimeUnit.MILLISECONDS);
        if (allDone) {
            IO.println("🎉 Tüm alt servisler yanıt verdi! Yanıt birleştirildi.");
        } else {
            IO.println("⚠ Bazı servisler zaman aşımına uğradı, fallback yanıtı dönülüyor!");
        }
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Kritik Mülakat Hatası: `countDown()` metodunu `try-finally` bloğu içine almamak! Eğer worker thread içinde beklenmedik bir RuntimeException fırlarsa ve `countDown()` çağrılamazsa, ana thread sonsuza dek `await()`te asılı kalır! Kural: `try { ... } finally { latch.countDown(); }`!

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Critical Interview Pitfall: Omitting `latch.countDown()` from a `finally` block! If a worker thread throws an unhandled RuntimeException before reaching `countDown()`, the counter never decrements to zero, hanging the coordinating thread permanently.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- CountDownLatch tek kullanımlıktır; sayaç sıfıra ulaştığında bir daha sıfırlanamaz.
- countDown() çağrıları thread çökmelerine karşı DAİMA finally içinde olmalıdır.
- Mikroservis paralel sorgulamalarında (Fan-Out/Fan-In) zaman aşımlı await kullanılır.

🎯 Key Takeaways for the Interview

- CountDownLatch is strictly single-use and cannot be reset once drained.
- Always wrap countDown() invocations inside a finally block to prevent hangs.
- Use await(timeout) for resilient SLA enforcement in parallel fan-out architectures.